OPERATING SYSTEMS LAB

Assignments 1 to 5 — Beginner Practical Code Book

SPPU TY AIML | Linux / Bash / C / POSIX Threads / IPC

Includes 3 practice programs per assignment, commented code, execution steps, and sample outputs.

Use this as a practical-exam preparation guide. Exact questions may differ from your college assignment sheet.

How to use this PDF

• Open Ubuntu (WSL) and create a lab folder: `mkdir -p ~/OS_Lab && cd ~/OS_Lab`.

• Save each program using the filename shown. For shell scripts, run `chmod +x filename.sh` and then `./filename.sh`.

• For C programs, compile with `gcc filename.c -o filename -pthread` when pthreads are used; run with `./filename`.

• Sample output is illustrative. Your output may vary depending on inputs and Linux version.

Assignment 1 — Linux Environment and File Management

Practice questions are organized into three likely practical tasks. Verify wording against your instructor's sheet.

1.1 Basic file and directory commands

Aim: Practice creating, listing, copying, moving, and deleting files/directories.

Code — a1_1.sh

```
#!/bin/bash
# Create a directory and move into it.
mkdir -p ~/OS_Lab/files
cd ~/OS_Lab/files || exit 1

# Create two files and write sample text.
echo "Operating Systems Lab" > notes.txt
echo "Linux commands practice" > second.txt

# Show current directory and contents.
pwd
ls -l

# Copy, rename, and display a file.
cp notes.txt backup.txt
mv second.txt renamed.txt
cat notes.txt

# Remove the copied file; -f avoids a prompt.
rm -f backup.txt
```

Steps: Save as a1_1.sh. Run: chmod +x a1_1.sh; ./a1_1.sh

Sample output:

```
/home/.../OS_Lab/files\n-rw-r--r-- ... notes.txt renamed.txt\nOperating Systems Lab
```

1.2 File permissions and ownership

Aim: Display and modify permissions using chmod; ownership change may require sudo.

Code — a1_2.sh

```
#!/bin/bash
# Create a file to demonstrate permissions.
touch permission_demo.txt
echo "Permission example" > permission_demo.txt

# Display current permissions.
ls -l permission_demo.txt

# Owner gets read/write; group and others get read only.
chmod 644 permission_demo.txt
ls -l permission_demo.txt

# Add execute permission for owner, then remove it.
chmod u+x permission_demo.txt
ls -l permission_demo.txt
chmod u-x permission_demo.txt
```

Steps: Save and run: chmod +x a1_2.sh; ./a1_2.sh

Sample output:

```
-rw-r--r-- ... permission_demo.txt\n-rwxr--r-- ... permission_demo.txt
```

1.3 Process monitoring

Aim: Use ps, top, and kill to inspect processes. The script safely launches and stops its own sleep process.

Code — a1_3.sh

```
#!/bin/bash
# Start a background process and remember its process ID.
sleep 60 &
pid=$!
echo "Started sleep with PID: $pid"

# Display process information.
ps -p "$pid" -o pid,ppid,stat,cmd

# Show a short snapshot of system processes.
ps -ef | head

# Stop only the process created by this script.
kill "$pid"
echo "Stopped PID: $pid"
```

Steps: Run: chmod +x a1_3.sh; ./a1_3.sh

Sample output:

```
Started sleep with PID: 1234\n PID PPID STAT CMD\n 1234 ... sleep 60\nStopped PID: 1234
```

Assignment 2 — Shell Scripting and Automation

Practice questions are organized into three likely practical tasks. Verify wording against your instructor's sheet.

2.1 Even/odd and prime number check

Aim: Read an integer, check parity, and test whether it is prime.

Code — a2_1.sh

```
#!/bin/bash
# Read an integer from the keyboard.
read -p "Enter an integer: " n

# Check even or odd using remainder.
if (( n % 2 == 0 )); then
    echo "$n is even"
else
    echo "$n is odd"
fi

# Values less than 2 are not prime.
if (( n < 2 )); then
    echo "$n is not prime"
    exit 0
fi

# Try divisors from 2 through n-1.
prime=1
for (( i=2; i<n; i++ )); do
    if (( n % i == 0 )); then
        prime=0
        break
    fi
done

if (( prime == 1 )); then
    echo "$n is prime"
else
    echo "$n is not prime"
fi
```

Steps: Run: chmod +x a2_1.sh; ./a2_1.sh; enter 7

Sample output:

```
Enter an integer: 7\n7 is odd\n7 is prime
```

2.2 Menu-driven file operations

Aim: Use functions and a menu to create/delete files and count lines, words, and characters.

Code — a2_2.sh

```
#!/bin/bash
# Each function performs one menu operation.
create_file() {
    read -p "File name: " file
    touch "$file"
    echo "File created: $file"
}
delete_file() {
    read -p "File name: " file
    if [[ -f "$file" ]]; then
        rm -- "$file"
        echo "File deleted"
    else
        echo "File not found"
    fi
}
count_file() {
    read -p "File name: " file
    if [[ -f "$file" ]]; then
        wc "$file"   # prints lines, words, bytes
    else
        echo "File not found"
    fi
}

# Repeat menu until user chooses Exit.
```

```
while true; do
    echo "1.Create  2.Delete  3.Count  4.Exit"
    read -p "Choice: " choice
    case "$choice" in
        1) create_file ;;
        2) delete_file ;;
        3) count_file ;;
        4) echo "Exiting"; break ;;
        *) echo "Invalid choice" ;;
    esac
done
```

Steps: Run in a practice directory: chmod +x a2_2.sh; ./a2_2.sh

Sample output:

```
1.Create  2.Delete  3.Count  4.Exit\nChoice: 1\nFile name: demo.txt\nFile created: demo.txt
```

2.3 Backup text files and log sizes

Aim: Find .txt files, copy them to a timestamped backup folder, and log names and sizes.

Code — a2_3.sh

```
#!/bin/bash
# Make a unique backup folder using date/time.
backup="backup_$(date +%Y%m%d_%H%M%S)"
mkdir -p "$backup"
log="$backup/backup_log.txt"

# Search only current directory for text files.
for file in ./*.txt; do
    # Skip when no .txt file exists.
    [[ -f "$file" ]] || continue

    # Copy file and record its size in bytes.
    cp -- "$file" "$backup/"
    size=$(wc -c < "$file")
    echo "$(basename "$file"), ${size} bytes" >> "$log"
done

echo "Backup folder: $backup"
echo "Backup log:"
cat "$log"
```

Steps: Create one or more .txt files first. Run: chmod +x a2_3.sh; ./a2_3.sh

Sample output:

```
Backup folder: backup_20261031_120000\nBackup log:\nnotes.txt, 25 bytes
```

Assignment 3 — Processes, System Calls, and Scheduling

Practice questions are organized into three likely practical tasks. Verify wording against your instructor's sheet.

3.1 Process creation using fork()

Aim: Create a child process, print process IDs, and wait for child completion.

Code — a3_1.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    // fork() creates a child process.
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child: fork returns 0.
        printf("Child: PID=%d, Parent PID=%d\n", getpid(), getppid());
    } else {
        // Parent: fork returns child's PID.
        printf("Parent: PID=%d, Child PID=%d\n", getpid(), pid);
        wait(NULL); // Wait for child to finish.
        printf("Child completed\n");
    }
    return 0;
}
```

Steps: Compile/run: gcc a3_1.c -o a3_1; ./a3_1

Sample output:

```
Parent: PID=2000, Child PID=2001\nChild: PID=2001, Parent PID=2000\nChild completed
```

3.2 FCFS CPU scheduling

Aim: Calculate waiting and turnaround time using First Come First Served order.

Code — a3_2.c

```
#include <stdio.h>
int main(void) {
    int n, bt[20], wt[20], tat[20];
    printf("Number of processes: ");
    scanf("%d", &n);

    // Enter burst time in arrival order (FCFS).
    for (int i = 0; i < n; i++) {
        printf("Burst time P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0; // First process waits zero time.
    for (int i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    float avg_wt = 0, avg_tat = 0;
    printf("Process  BT  WT  TAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
        printf("P%d       %d   %d   %d\n", i+1, bt[i], wt[i], tat[i]);
    }
    printf("Average WT = %.2f\n", avg_wt/n);
    printf("Average TAT = %.2f\n", avg_tat/n);
    return 0;
}
```

Steps: Compile/run: gcc a3_2.c -o a3_2; ./a3_2

Sample output:

```
Number of processes: 3\nBurst time P1: 5\nBurst time P2: 3\nBurst time P3: 2\nProcess  BT  WT  TAT\nP1       5   0   5\nP2
```

3.3 Round Robin scheduling

Aim: Simulate Round Robin using a time quantum and calculate waiting/turnaround times.

Code — a3_3.c

```
#include <stdio.h>
int main(void) {
    int n, bt[20], rem[20], wt[20]={0}, tat[20];
    int quantum, time=0, done;

    printf("Number of processes: ");
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        printf("Burst time P%d: ", i+1);
        scanf("%d", &bt[i]);
        rem[i]=bt[i]; // Remaining burst time.
    }
    printf("Time quantum: ");
    scanf("%d", &quantum);

    // Give each unfinished process one quantum per round.
    do {
        done=1;
        for (int i=0; i<n; i++) {
            if (rem[i] > 0) {
                done=0;
                int run = rem[i] < quantum ? rem[i] : quantum;
                time += run;
                rem[i] -= run;
                if (rem[i] == 0)
                    tat[i]=time; // All processes arrive at time 0.
            }
        }
    } while (!done);

    float avgwt=0, avgtat=0;
    printf("Process  BT  WT  TAT\n");
    for (int i=0; i<n; i++) {
        wt[i]=tat[i]-bt[i];
        avgwt+=wt[i]; avgtat+=tat[i];
        printf("P%d       %d   %d   %d\n",i+1,bt[i],wt[i],tat[i]);
    }
    printf("Average WT=%.2f, Average TAT=%.2f\n",avgwt/n,avgtat/n);
    return 0;
}
```

Steps: Compile/run: gcc a3_3.c -o a3_3; ./a3_3

Sample output:

```
For burst times 5,3,2 and quantum 2: completion times (TAT) 10, 9, 6; WT 5, 6, 4
```

Assignment 4 — Multithreading and Resource Utilization

Practice questions are organized into three likely practical tasks. Verify wording against your instructor's sheet.

4.1 Create two POSIX threads

Aim: Create threads that print messages; main waits for both using pthread_join.

Code — a4_1.c

```
#include <stdio.h>
#include <pthread.h>

void *worker(void *arg) {
    int id = *(int *)arg;
    printf("Thread %d is running\n", id);
    return NULL; // Thread ends.
}

int main(void) {
    pthread_t t1, t2;
    int id1=1, id2=2;

    // Create two independent threads.
    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);

    // Wait so main does not exit early.
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("All threads completed\n");
    return 0;
}
```

Steps: Compile/run: gcc a4_1.c -o a4_1 -pthread; ./a4_1

Sample output:

```
Thread 1 is running\nThread 2 is running\nAll threads completed
```

4.2 Parallel sum of an array

Aim: Split an array into two parts and calculate each partial sum in a separate thread.

Code — a4_2.c

```
#include <stdio.h>
#include <pthread.h>

int a[]={1,2,3,4,5,6};
int sum1=0, sum2=0;

void *first_half(void *unused) {
    for (int i=0; i<3; i++) sum1 += a[i];
    return NULL;
}
void *second_half(void *unused) {
    for (int i=3; i<6; i++) sum2 += a[i];
    return NULL;
}
int main(void) {
    pthread_t t1,t2;
    pthread_create(&t1,NULL,first_half,NULL);
    pthread_create(&t2,NULL,second_half,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);

    // Combine the partial results after both threads finish.
    printf("First half sum = %d\n",sum1);
    printf("Second half sum = %d\n",sum2);
    printf("Total sum = %d\n",sum1+sum2);
    return 0;
}
```

Steps: Compile/run: gcc a4_2.c -o a4_2 -pthread; ./a4_2

Sample output:

```
First half sum = 6\nSecond half sum = 15\nTotal sum = 21
```

4.3 Mutex-protected shared counter

Aim: Use a mutex to protect a shared counter from simultaneous thread updates.

Code — a4_3.c

```
#include <stdio.h>
#include <pthread.h>

#define COUNT 100000
long counter=0;
pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg) {
    for (int i=0; i<COUNT; i++) {
        // Lock before changing shared data.
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
int main(void) {
    pthread_t t1,t2;
    pthread_create(&t1,NULL,increment,NULL);
    pthread_create(&t2,NULL,increment,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    printf("Final counter = %ld\n",counter);
    pthread_mutex_destroy(&lock);
    return 0;
}
```

Steps: Compile/run: gcc a4_3.c -o a4_3 -pthread; ./a4_3

Sample output:

```
Final counter = 200000
```

Assignment 5 — Interprocess Communication (IPC)

Practice questions are organized into three likely practical tasks. Verify wording against your instructor's sheet.

5.1 Pipe communication

Aim: Send a message from parent to child using an unnamed pipe.

Code — a5_1.c

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(void) {
    int fd[2]; // fd[0] read end, fd[1] write end.
    char message[]="Hello from parent";
    char buffer[100]={0};

    if (pipe(fd)==-1) { perror("pipe"); return 1; }
    pid_t pid=fork();

    if (pid==0) {
        close(fd[1]); // Child only reads.
        read(fd[0],buffer,sizeof(buffer));
        printf("Child received: %s\n",buffer);
        close(fd[0]);
    } else {
        close(fd[0]); // Parent only writes.
        write(fd[1],message,strlen(message)+1);
        close(fd[1]);
        wait(NULL);
    }
    return 0;
}
```

Steps: Compile/run: gcc a5_1.c -o a5_1; ./a5_1

Sample output:

```
Child received: Hello from parent
```

5.2 Shared memory communication

Aim: Create shared memory with mmap so parent and child can access the same message.

Code — a5_2.c

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>

int main(void) {
    // MAP_SHARED memory is visible to parent and child after fork.
    char *shared=mmap(NULL,4096,PROT_READ|PROT_WRITE,
                      MAP_SHARED|MAP_ANONYMOUS,-1,0);
    if (shared==MAP_FAILED) { perror("mmap"); return 1; }

    pid_t pid=fork();
    if (pid==0) {
        strcpy(shared,"Message written by child");
        _exit(0);
    } else {
        wait(NULL); // Ensure child writes before parent reads.
        printf("Parent reads: %s\n",shared);
        munmap(shared,4096);
    }
    return 0;
}
```

Steps: Compile/run: gcc a5_2.c -o a5_2; ./a5_2

Sample output:

```
Parent reads: Message written by child
```

5.3 Message queue using POSIX message queues

Aim: Send and receive a short message using a named POSIX message queue.

Code — a5_3.c

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

int main(void) {
    const char *name="/oslab_queue";
    struct mq_attr attr={0};
    attr.mq_maxmsg=10;
    attr.mq_msgsize=128;

    // Create queue; remove an old queue first for repeatable practice.
    mq_unlink(name);
    mqd_t q=mq_open(name,O_CREAT|O_RDWR,0644,&attr);
    if (q==(mqd_t)-1) { perror("mq_open"); return 1; }

    char msg[]="IPC message";
    mq_send(q,msg,strlen(msg)+1,0);

    char buffer[128];
    unsigned priority;
    ssize_t bytes=mq_receive(q,buffer,sizeof(buffer),&priority);
    if (bytes>=0) printf("Received: %s\n",buffer);
    else perror("mq_receive");

    mq_close(q);
    mq_unlink(name); // Delete queue name.
    return 0;
}
```

Steps: Compile/run: gcc a5_3.c -o a5_3 -lrt; ./a5_3

Sample output:

```
Received: IPC message
```

Quick viva revision

Q. What is a process?
Ans. A program in execution, with its own process ID and resources.

Q. What does fork() do?
Ans. Creates a child process; returns 0 in child and child's PID in parent.

Q. What is FCFS?
Ans. A non-preemptive scheduling method that executes processes in arrival order.

Q. What is Round Robin?
Ans. A preemptive scheduling method that gives each ready process a fixed time quantum.

Q. What is a thread?
Ans. A lightweight execution unit within a process, sharing process resources.

Q. Why use a mutex?
Ans. To protect shared data from simultaneous access and race conditions.

Q. What is IPC?
Ans. Interprocess communication: exchanging data between processes.

Q. Pipe vs shared memory?
Ans. Pipe transfers data through a kernel buffer; shared memory maps the same memory region into processes.

Q. What is chmod?
Ans. Changes file permission bits.

Q. What does wait() do?
Ans. Makes the parent wait for a child process to finish.

Exam reminders

• Use gcc filename.c -o program for ordinary C programs.

• Use gcc filename.c -o program -pthread for pthread programs.

• For scripts: chmod +x file.sh, then ./file.sh.

• Explain the aim, important system call/function, and output to the examiner.

• Do not run destructive commands on important files. Practice inside ~/OS_Lab.